Adaptive LLM Routing for Mathematical Reasoning

COMP 345: Midterm Report

Ziyad Kazi Varshini Loganathan

October 30, 2026

GitHub repository: *Insert repository URL before submission.*
Demo notebook: `MATH_Adaptive_Routing.ipynb`

1 Problem

Larger language models can be more capable, but they are often more expensive to use. Many questions can also be answered correctly by a smaller model. Our project studies whether we can choose the cheaper model when it is sufficient and use the larger model when needed.

We aim to make this decision using only the question, before either language model generates an answer. For the midterm, we built a working pipeline that collects model responses, grades their answers, and compares accuracy, estimated cost, and latency.

2 Hypothesis

We hypothesize that a lightweight classifier can use a frozen question embedding to predict whether a smaller model will answer correctly. This could reduce inference costs while maintaining accuracy close to always using the larger model.

Our target is to remain within **three percentage points** of the larger model's accuracy while using no more than **75% of its estimated inference cost**. We have completed the initial baseline study. Training and evaluating the router on real model outputs is the next stage.

3 Related Approaches

FrugalGPT studies cascades that use multiple language models to balance cost and answer quality [1]. RouteLLM learns to route queries between stronger and weaker models using preference data [2]. Hybrid LLM also routes queries based on predicted difficulty and a desired quality level [3].

Our project follows this general approach. We focus on predicting correctness on mathematics problems using frozen question embeddings and a simple classifier. We will compare against basic routing strategies to determine whether the learned classifier provides a useful improvement.

4 Dataset

Our original proposal used GSM8K. Based on instructor feedback that GSM8K may be too easy for current models, we switched to **MATH**, starting with its algebra subset [4].

For the pilot, we evaluated 50 questions from a seeded random sample of the official training split, using seed 345. The questions include different algebra topics and answer formats, such as numbers, complex expressions, and intervals.

We used the provided solutions to grade model responses. The correct solutions were never included in the prompts sent to the models. This pilot is a development study, not a final held-out evaluation.

5 Models and Workflow

5.1 Implemented Baseline System

We used two models through the Groq API:

- **Smaller model:** GPT-OSS-20B.
- **Larger model:** GPT-OSS-120B.

These replaced the originally proposed Llama configurations so that we could run the pilot using free API access.

Both models received the same question and instructions to provide a concise explanation and place their final answer inside `\boxed{...}`. We used a temperature of 0 and a maximum completion budget of 2,048 tokens.

For each question, the implemented workflow was:

1. Send the problem to both models.
2. Save each response, token usage, completion status, and request latency.
3. Extract the final boxed answer.
4. Compare it with the dataset answer using Math-Verify [5].
5. Review unresolved grading cases and calculate baseline results.

Responses were saved to Google Drive after each request. Completed requests were skipped when collection resumed, preventing unnecessary repeat calls.

5.2 Planned Router

The router will take a frozen question embedding and predict a correctness probability for each model. We plan to use the locally run embedding model `all-MiniLM-L6-v2` [6].

For a question embedding x, the output for model k will be

$$p_k = \sigma(w_k^\top x + b_k),$$

where σ is the sigmoid function. Each output has its own weights and bias. The training label y_k is 1 if model k answered correctly and 0 otherwise.

We will train the router using binary cross-entropy loss:

$$\mathcal{L} = -\frac{1}{2N} \sum_{i=1}^{N} \sum_{k=1}^{2} \left[y_{ik} \log p_{ik} + (1 - y_{ik}) \log(1 - p_{ik}) \right].$$

Adam will update only the router's weights. The embedding model and both language models will remain frozen.

At inference time, the routing rule will be

$$r(x) = \begin{cases} \text{small model}, & p_{\text{small}}(x) \geq \tau, \\ \text{large model}, & p_{\text{small}}(x) < \tau. \end{cases}$$

With two models and an always-large fallback, only the small model's predicted probability determines the routing decision.

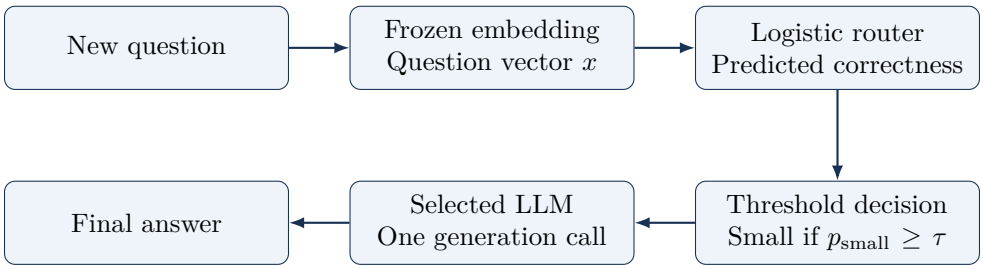


Figure 1: Planned inference workflow. The router selects a model before either language model generates an answer. The current pilot evaluates the two models separately.

6 Experiments

The pilot compared two baselines: always using the smaller model and always using the larger model. We also calculated oracle accuracy, which assumes we could select a correct model whenever either model answered successfully.

We measured:

- Final-answer accuracy.
- Input and output token counts.
- Estimated inference cost.
- Mean request latency.
- Agreement and disagreement in model correctness.

For each model call, estimated cost was calculated as

$$C = \frac{T_{\text{in}} P_{\text{in}} + T_{\text{out}} P_{\text{out}}}{10^6},$$

where T denotes token count and P denotes the price in dollars per million tokens.

These estimates use standard uncached pricing assumptions [7]; they are not actual charges. Our API spending was zero because we used the free tier. Neither baseline requires an embedding call.

Table 1: Pricing assumptions used to estimate deployment cost.

Model	Input / 1M tokens	Output / 1M tokens
GPT-OSS-20B	$0.075	$0.30
GPT-OSS-120B	$0.15	$0.60

The next experiment will compare the learned router with always-small, always-large, random routing, and a question-length heuristic. We will select the routing threshold on validation data and evaluate the final policy on separate test questions. Router evaluation will also account for embedding overhead.

7 Results

Table 2 summarizes the pilot results.

Table 2: Baseline results on 50 MATH algebra questions.

Metric	GPT-OSS-20B	GPT-OSS-120B
Correct answers	49/50	50/50
Accuracy	98%	100%
Mean input tokens	162.98	162.98
Mean output tokens	577.40	476.62
Mean request latency	0.855 s	1.275 s
Estimated total cost	$0.009272	$0.015521

Both models answered 49 questions correctly. Only the larger model successfully answered the remaining question. No question was unsuccessful for both models, so oracle accuracy was 100%.

Always using the smaller model reduced estimated cost by **40.3%**, with an accuracy decrease of **two percentage points**. Therefore, the always-small baseline already met both target conditions on this sample.

The smaller model generated more output tokens on average, but its lower token rates still made it cheaper. These savings came from using the smaller model directly, not from a learned routing policy.

8 Failure Analysis

The only unsuccessful response came from the smaller model on a problem involving repeated application of a quadratic function. The model found the correct four real solutions but reached the completion-token limit before finishing their sum or providing a final answer.

We counted this as unsuccessful under the fixed generation budget and classified it as a **truncation failure**. The response does not clearly demonstrate that the model was unable to solve the mathematics.

This question also contained Asymptote source code for a graph. The models received that code as text rather than seeing a rendered image. Performance on this example therefore does not measure visual graph understanding.

9 Discussion and Limitations

The pilot shows that the smaller model was already sufficient for nearly all sampled questions. Although we switched from GSM8K to MATH, this algebra sample still provided little room for a router to improve accuracy.

A learned router needs to improve on the always-small baseline, rather than only save money compared with always-large. With just one observed small-model failure, we do not yet have enough failure examples to learn a reliable routing rule.

The study is limited by its size and scope. With 50 questions, one answer changes accuracy by two percentage points. These results do not establish that the same performance will hold across the full MATH dataset. Automated grading may introduce errors, and the latency measurements reflect one collection run. The measured request times exclude the deliberate pauses between API calls.

Before the final report, we plan to expand the development sample across subjects and difficulty levels, collect more correctness labels, and train the router. Training, validation, and final test questions will remain separate. We will also measure embedding overhead and examine whether routing errors are associated with question length, difficulty, or response truncation.

References

[1] L. Chen, M. Zaharia, and J. Zou. *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance*. 2023. https://arxiv.org/abs/2305.05176

[2] I. Ong et al. *RouteLLM: Learning to Route LLMs with Preference Data*. 2024. https://arxiv.org/abs/2406.18665

[3] D. Ding et al. *Hybrid LLM: Cost-Efficient and Quality-Aware Query Routing*. 2024. https://arxiv.org/abs/2404.14618

[4] EleutherAI. *MATH Dataset: Hugging Face Distribution*. https://huggingface.co/datasets/EleutherAI/hendrycks_math

[5] Hugging Face. *Math-Verify*. https://github.com/huggingface/Math-Verify

[6] Sentence Transformers. *all-MiniLM-L6-v2 Model Card*. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

[7] Groq. *Supported Models and Pricing*. https://console.groq.com/docs/models